

Scooter Parking

Time Limit: 1 second

Memory Limit: 128 MB

Problem Description

Scooter parking areas near NCKU are always packed with scooters lining up along a straight parking strip. To avoid collisions, each arriving student aims to maximize the distance to the nearest scooter by choosing the largest available gap.

Initially, n scooters are parked along the strip, forming a row where **each scooter is closer to the one on its right than to the one on its left**, except for the first and the last scooters, which only have one neighbor. In other words, if we let $p_1 < p_2 < \dots < p_n$ denote the positions of the scooters and $g_i = p_{i+1} - p_i$ denote the gap between the i -th and $(i + 1)$ -th scooters, then $g_1 > g_2 > \dots > g_{n-1}$.

Each position is given as a real number with **at most one digit after the decimal point**, and that digit (if present) is guaranteed to be **0 or 5**. In other words, $2p_i$ is always an integer.

Then, m more scooters arrive one at a time, and each newly arrived scooter follows this rule:

1. Find the **largest gap** between two adjacent scooters currently on the strip. A gap between two adjacent scooters at positions l and r (where $l < r$) has length $r - l$.
2. If multiple gaps tie for the largest length, choose the **leftmost** one.
3. Park the new scooter **exactly in the middle** of the chosen gap (l, r) , i.e., at position $\frac{l + r}{2}$, splitting it into two equal smaller gaps.

A Warm-up Example

Suppose the initial positions are $[0, 6, 8]$ (gaps $6 > 2$, satisfying the decreasing rule above).

- Round 1: the largest gap is $(0, 6)$, so the first new scooter parks at $(0 + 6)/2 = 3$, at distance 3 from its nearest scooter. The strip becomes $[0, 3, 6, 8]$.
- Round 2: the gaps are $(0, 3) = 3$, $(3, 6) = 3$, $(6, 8) = 2$. The first two tie for largest; choose the leftmost, $(0, 3)$. The new scooter parks at $(0 + 3)/2 = 1.5$, at distance 1.5 from its nearest scooter. The strip becomes $[0, 1.5, 3, 6, 8]$.

Try continuing this by hand for one more scooter before coding, to make sure you understand the rule.

Input

The first line contains two integers n and m , representing the number of initially parked scooters and the number of arriving scooters, respectively.

The second line contains n space-separated real numbers $p_1, p_2, \dots, p_n$, where p_i denotes the initial position of the i -th scooter. Each p_i is printed with **exactly one digit after the decimal point** (e.g., **5.0** or **5.5**).

Output

Output m lines. The k -th line should contain a single real number — the distance between the k -th newly arrived scooter and its nearest scooter, measured immediately after it parks, **rounded to exactly 6 digits after the decimal point** (e.g., `printf("%.6f\n", ans)` in C).

Constraints

- $2 \leq n \leq 2 \times 10^5$
- $1 \leq m \leq 2 \times 10^5$
- $0 \leq p_1 < p_2 < \dots < p_n \leq 10^9$, and $2p_i \in \mathbb{Z}$ for every i
- $g_1 > g_2 > \dots > g_{n-1}$, where $g_i = p_{i+1} - p_i$

Hint

- Whenever a gap (l, r) is split, both new gaps have length $(r - l)/2$ — neither is larger than the original, and the two are always tied. Combined with the fact that the initial gaps are already given in decreasing order, this means you can always track the current largest gap using **queues** instead of a more complex data structure (like a max-heap).

Sample Testcase 1

Sample Input 1

```
3 3
0.0 6.0 8.0
```

Sample Output 1

3.000000

1.500000

1.500000

Explanation of Sample 1

- Round 1: Gaps are $(0, 6) = 6$ and $(6, 8) = 2$. Largest is $(0, 6)$. Park at $(0 + 6)/2 = 3$, distance to nearest scooter is 3 (**output** 3.000000). strip: $[0, 3, 6, 8]$.
- Round 2: Gaps are $(0, 3) = 3$, $(3, 6) = 3$, $(6, 8) = 2$. The first two tie for largest; choose leftmost $(0, 3)$. Park at $(0 + 3)/2 = 1.5$, distance to nearest scooter is 1.5 (**output** 1.500000). strip: $[0, 1.5, 3, 6, 8]$.
- Round 3: Gaps are $(0, 1.5) = 1.5$, $(1.5, 3) = 1.5$, $(3, 6) = 3$, $(6, 8) = 2$. Largest is $(3, 6) = 3$. Park at $(3 + 6)/2 = 4.5$, distance to nearest scooter is $\min(4.5 - 3, 6 - 4.5) = 1.5$ (**output** 1.500000). strip: $[0, 1.5, 3, 4.5, 6, 8]$.